LAB12: TREES

#WAP to illustrate the concept of tree data structure and its operations

```
class Node:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right =None


class BST:
    def __init__(self):
        self.root=None


    def insert(self,data):
        newnode=Node(data)
        curnode=None
        if self.root is None:
            self.root=newnode
        else:
            curnode=self.root
        while curnode!=None:
            if data < curnode.data:
                if curnode.left is None:
                    curnode.left=newnode
                    break
                else:
```

```
                curnode = curnode.left
            else:
                if curnode.right is None:
                    curnode.right=newnode
                    break
                else:
                    curnode=curnode.right

    def search(self,key):
        curnode = self.root
        while curnode is not None:
            if key == curnode.data:
                return True
            elif key < curnode.data:
                curnode = curnode.left
            else:
                curnode=curnode.right
        return False
    def inorder(self, rt):
        if rt.left is not None:
            self.inorder(rt.left)
        print(rt.data, end="\t")
        if rt.right is not None:
            self.inorder(rt.right)
```

```
def preorder(self, rt):
    print(rt.data, end="\t")
    if rt.left is not None:
        self.preorder(rt.left)
    if rt.right is not None:
        self.preorder(rt.right)

def postorder(self, rt):
    if rt.left is not None:
        self.postorder(rt.left)
    if rt.right is not None:
        self.postorder(rt.right)
    print(rt.data, end="\t")

def delete(self,key):
    curnode = self.root
    parentnode =None
    while curnode is not None:
        if key == curnode.data:
            if temp=="Left":
                parentnode.left=None
            else:
                parentnode.right=None
            print(key,"Node Deleted")
            return True
```

```
            elif key < curnode.data:
                parentnode = curnode
                curnode = curnode.left
                temp="Left"
            else:
                parentnode = curnode
                curnode=curnode.right
                temp="Right"
        print(key,"Node not found")
        return False
b = BST()
L= [25,10,35,20,65,45,24]
while True:
    print("\n1.insert")
    print("2.delete")
    print("3.search")
    print("4.inorder")
    print("5.preorder")
    print("6.postorder")
    print("7.Exit")
    ch=int(input("Enter ur choice:"))
    if ch==1:
          for i in L:
              b.insert(i)
```

```
    elif ch==2:
            b.delete(24)
    elif ch==3:
            key=int(input("Enter the element to be searched:"))
            print(f"{key} found:{b.search(key)}")
    elif ch==4:
        print("\nInorder Traversal")
        b.inorder(b.root)
    elif ch==5:
        print("\nPreorder Traversal")
        b.preorder(b.root)
    elif ch==6:
        print("\nPostorder Traversal")
        b.postorder(b.root)
    else:
        break
```

OUTPUT:

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:1

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:4

Inorder Traversal

10 20 24 25 35 45 65

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:5

Preorder Traversal

25 10 20 24 35 65 45

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:6

Postorder Traversal

24 20 10 45 65 35 25

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:3

Enter the element to be searched:45

45 found:True

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:2

24 Node Deleted

1.insert

2.delete

3.search

4.inorder

5.preorder

6.postorder

7.Exit

Enter ur choice:7